CSAI 203 Course Project Fall-2026

Zewail City of Science, Technology and Innovation

University of Science and Technology

School of Computational Sciences and Artificial Intelligence

CSAI 203 - Fall 2026

Introduction to Software Engineering

# Academic Course & Schedule Planner Software Requirements Specification (SRS)

**Team Members**:

Seif Ahmed Abouelkhair – 202401936

Mohab Mohamed – 202401064

Eyad Mohamed – 202402463

**Representative Contact:**

s-seif.abouelkhair@zewailcity.edu.eg

**Date:** 30 October 2026

**Table of Contents :**

1. Introduction (3)

1.1 Purpose (3)

1.2 Scope (3)

1.3 Definitions, Acronyms, and Abbreviations (4)

1.4 References ( 5 )

1.5 Overview (6)

2. Overall Description ( 6)

2.1 Product Perspective (6)

2.2 Product Functions (7)

2.3 User Classes and Characteristics (8)

2.4 Software Stakeholders (8)

2.5 Operating Environment (9)

2.6 Design and Implementation Constraints (10)

2.7 User Documentation (11)

2.8 Assumptions and Dependencies (12)

3. Specific Requirements (13)

3.1 Functional Requirements (13-18)

3.2 Use Case Model (19)

3.2.1 Use Case Diagrams (19–21)

3.2.2 Use Case Descriptions (22_50)

3.3 Domain Model (50)

3.3.1 Conceptual Class Diagram (51)

3.3.2 Class Descriptions (52_54)

3.4 Non-Functional Requirements ( 55_58)

3.5 External Interface Requirements (58_60)

4. Appendices (61)

4.1 Data Dictionary ( 62)

4.2 Glossary (63_65)

## 1. Introduction :_

### 1.1 Purpose

This Software Requirements Specification (SRS) defines the expected behavior, constraints, interfaces, and quality requirements of the Academic Course & Schedule Planner. It provides a shared reference for the student team, course instructor, designers, developers, and testers. The document will be used to guide system design and implementation, and trace each implemented feature back to an agreed requirement. It describes what the system must do rather than the detailed internal code structure.

### 1.2 Scope

The Academic Course & Schedule Planner is a browser-based application that helps university students plan an academic semester. Students can explore available courses and sections, review prerequisites, create a conflict-free timetable, maintain a list of selected courses, and estimate semester and  CGPA. Administrators maintain the academic information on which planning depends, including courses, section offerings, capacities, prerequisites, and academic grading rules.

The system will also provide a visual academic progress dashboard using charts, graphs, and progress indicators. Students will be able to visualize their completed credit hours, remaining credit hours, academic progress toward graduation, and other relevant academic statistics. These visualizations will help students understand their current academic standing, monitor their progress, and make more informed decisions when planning future semesters.

The system is intended as an academic planning and information-management tool. Unless a future integration is approved, it will not officially register students with the university, guarantee seat availability, replace the official student information system, or make graduation decisions. Its schedule, GPA, and academic progress outputs are planning estimates based on the data entered or maintained in the application.

### 1.3 Definitions, Acronyms, and Abbreviations

| | |
|---|---|
| Administrator (Admin) | Authorized user who maintains course, section, capacity, prerequisite, and academic-rule data. |
| Course | A catalog subject identified by a unique code, with a title, credit hours, and prerequisite rules. |
| Section / Offering | A particular scheduled instance of a course in a specific semester, with meeting times, instructor, and capacity. |
| Prerequisite | A course or academic condition that must be satisfied before a student may take another course. |
| Schedule conflict | An overlap between the meeting times of two selected sections. |
| C/ GPA | Cumulative/ Grade Point Average, calculated from course credits and grade points. |
| SRS | Software Requirements Specification. |
| UML | Unified Modeling Language, used to model system behavior and structure. |
| CRUD | Create, Read, Update, and Delete operations on stored data. |
| FR / NFR | Functional Requirement / Non-Functional Requirement. |
| HTTPS | Hypertext Transfer Protocol Secure, used for encrypted communication. |

### 1.4 References

Institute of Electrical and Electronics Engineers. (2018). *Systems and software engineering—Life cycle processes—Requirements engineering* (IEEE Std 29148-2018). IEEE.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Perforce Software. (2025). *How to write a software requirements specification (SRS) document*. https://www.perforce.com/blog/alm/how-write-software-requirements-specification-srs-document

Zewail City of Science, Technology and Innovation. (2026). *CSAI 203: Introduction to Software Engineering—Fall 2026 project instructions and SRS outline*

### 1.5 Overview

Section 2 introduces the product context, users, stakeholders, environment, constraints, documentation, and assumptions. Section 3 specifies the system's functional requirements, use cases, conceptual domain model, non-functional requirements, and external interfaces. Section 4 provides a preliminary data dictionary and glossary. Requirement identifiers (FR-01 to FR-20 and NFR-01 to NFR-05) are used consistently to support later design, implementation, and testing.

---

## 2. Overall Description

## 2.1 Product Perspective

The product is a standalone web application for academic planning. It consists of a browser-based user interface, an application layer that validates planning rules and performs calculations, and a persistent database that stores users, academic. Administrators enter or update the data required by students; students consume this data to plan their semesters

## 2.2 Product Functions

The Academic Course & Schedule Planner will provide the following major functions:

- **Account Management:** Account registration, login, logout, and role-based access for students and administrators.
- **Course Catalog:** Course catalog browsing, keyword search, and filtering by department, credit hours, and semester availability.
- **Course Information:** Course detail display, including credit hours, course descriptions, prerequisites, and available sections.
- **Prerequisite Checking:** Automatic prerequisite eligibility checking using a student's recorded completed courses before allowing a course to be added to a semester plan.
- **Section Selection and Schedule Generation:** Section selection, automatic timetable generation, schedule conflict detection, and suggestions for alternative sections when conflicts occur.
- **Semester Plan Management:** Saving, editing, and reviewing semester plans, including adding and removing selected courses and sections.
- **GPA Calculation:** Semester GPA and cumulative GPA estimation using configurable academic grading rules.
- **Academic Progress Tracking:** Tracking completed courses, earned credit hours, remaining credit hours, and unmet prerequisite requirements.
- **Visual Academic Dashboard:** Providing interactive charts, graphs, and progress indicators to visualize academic performance and progress toward graduation. This includes visual representations of completed versus remaining credit hours, semester GPA trends, cumulative GPA changes, and course completion statistics.
- **Graduation Progress Overview:** Displaying an estimated overview of a student's progress toward completing their degree requirements, helping them understand how many credit hours and courses remain.
- **Administrative Management:** Administrative management of courses, course offerings, section capacities, prerequisites, grading rules, and academic reports.
- **Data and Report Management:** Generating and displaying academic planning reports and summaries to help students review their semester plans and academic progress.

## 2.3 User Classes and Characteristics

**Students:** The primary users. They are expected to have basic web and computer literacy. They may have different levels of familiarity with university course codes and prerequisite policies. The interface should use clear labels, explanations, and validation messages. Students can access only their own saved plans and academic progress.

**Administrators:** Users responsible for maintaining accurate academic data. They are expected to understand the institution's course catalog, semester offerings, prerequisite policies, and capacity rules. They require forms that support accurate data entry, confirmation before destructive changes, and clear feedback when data conflicts with existing plans.

## 2.4 Software Stakeholders

| Stakeholder | Interest |
|---|---|
| **Students** | Accurate course information, easy schedule planning, prerequisite visibility, reliable GPA estimates, and visual representations of completed and remaining credit hours and academic progress. |
| **Academic Advisors** | Students' semester plans should be understandable and consistent with academic rules. The system may support advising by providing academic progress summaries and identifying unmet requirements, but it does not replace professional academic advising. |
| **Registrar Staff** | Correct course catalog, section offerings, capacities, prerequisites, degree requirements, and grading policies. They also need controlled administrative access to maintain academic information. |
| **Course Instructors** | Accurate course descriptions, meeting times, credit hours, and section information to reduce confusion regarding course offerings. |
| **University Administration** | Reliable academic information and summarized reports that may help monitor academic planning and student progress. |
| **Developers** | Clear requirements, manageable implementation scope, maintainable software architecture, and testable acceptance criteria. |
| **Course Instructor** | Evidence that the requirements, models, design, implementation, and testing activities are consistent and satisfy the project specifications. |

### 2.5 Operating Environment

The Academic Course & Schedule Planner will operate as a browser-based web application, allowing students and administrators to access the system through devices connected to the internet without requiring dedicated desktop software installation.

The system will operate in the following environment:

- Client Devices: Desktop computers, laptops, tablets, and smartphones with sufficient resources to run a modern web browser.
- Operating Systems: Windows, macOS, Linux, Android, and iOS.
- Web Browsers: The application will support modern versions of Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari.
- Network Requirements: A stable internet connection will be required to access the application, retrieve academic information, save semester plans, and update user data.
- Server Environment: The application will run on a web server responsible for handling user requests, processing academic information, performing GPA calculations, checking schedule conflicts, and communicating with the database.
- Database Environment: A relational database management system will be used to store and manage user accounts, course information, section offerings, prerequisites, semester plans, and academic records.
- User Interface: The application will provide a responsive interface that adapts to different screen sizes and supports interactive charts, graphs, and academic progress visualizations.

### 2.6 Design and Implementation Constraints

The development of the Academic Course & Schedule Planner will be subject to the following design and implementation constraints:

- **Web-Based Application:** The system must be developed as a browser-based application rather than a standalone desktop application.
- **Programming Technologies:** The development team will use agreed-upon programming languages, frameworks, and tools suitable for web application development.
- **Database Constraints:** The system must use a structured database to store and manage user accounts, courses, section offerings, prerequisites, semester plans, and academic records while maintaining data consistency.
- **Security Requirements:** User authentication and role-based access control must be implemented to ensure that students and administrators can only access the functions permitted for their roles. Passwords must not be stored in plain text.
- **Academic Rules:** GPA calculations, prerequisite checking, and academic progress tracking must follow configurable academic rules. The system must not assume that its calculations represent official university records unless verified.
- **Browser Compatibility:** The application must support modern web browsers and provide a responsive interface for different screen sizes.
- **Project Scope and Time:** The implementation must remain within the project requirements and the available development time, prioritizing the required student and administrator functionalities.
- **Data Privacy:** Student academic information and account details must be protected and accessible only to authorized users.

### 2.7 User Documentation

The project will provide a short FAQ and user guides to help students and administrators understand and use the system effectively.

The student guide will explain account access, searching and filtering the course catalog, checking prerequisites, selecting courses and sections, building and saving a conflict-free timetable, interpreting schedule conflict messages, and using semester and cumulative GPA calculations. It will also explain how to read the academic dashboard, interpret charts and progress indicators, and track completed and remaining credit hours.

The administrator guide will explain how to manage courses and section offerings, update section capacities, maintain prerequisite information, configure grading rules, and generate academic reports.

Additionally, the system will provide basic validation messages, error notifications, and contextual guidance within the interface to help users understand their actions and resolve common issues.

### 2.8 Assumptions and Dependencies

The development and operation of the Academic Course & Schedule Planner will depend on the following assumptions:

- Users have access to a supported web browser and a working internet connection.
- Administrators are responsible for providing accurate and updated course, section, meeting-time, capacity, prerequisite, degree-requirement, and grading-scale information.
- The application uses a configured academic term and institution-specific grading scale. Until the official rules are confirmed, GPA calculation rules will remain configurable rather than hard-coded as official university policy.
- Completed courses and grades are either entered by students or supplied by an authorized administrator. Automatic transcript import is not assumed in the initial version.
- Students are responsible for ensuring that their entered academic records are accurate if the information is not provided by an authorized university source.
- The initial version does not connect to the university's official registration, learning management, or identity-management systems.
- The system depends on a properly configured database to store and retrieve user accounts, academic information, semester plans, and student progress records.
- Email verification and password-reset email delivery may depend on an external email service if these features are implemented.
- Academic progress and graduation estimates depend on the availability and accuracy of the university's degree requirements and credit-hour policies.

## 3. Specific Requirements

### 3.1 Functional Requirements

The following functional requirements define the expected behavior and capabilities of the Academic Course & Schedule Planner. Each requirement has a unique identifier and describes the system's expected operations, processing rules, and validation behavior. Unless otherwise stated, the system shall save successful changes and display an appropriate confirmation or error message to the user.

**FR-01 – User Registration**

The system shall allow new students to create accounts by providing the required personal and login information, including full name, university email address, and password. It shall validate all required fields, reject email addresses that are already registered, and enforce the configured password policy. Passwords shall be stored securely using password hashing rather than plain text. Upon successful registration, the system shall create a Student account and display a confirmation message. If email verification is enabled, the account shall remain pending until the verification process is completed.

**FR-02 – Login and Logout**

The system shall allow registered users to access their accounts using their email addresses and passwords. Upon successful authentication, the system shall create a secure session and redirect the user to the appropriate dashboard based on their assigned role. Invalid login attempts shall display a general error message without revealing whether the entered email address exists. Users shall also be able to log out securely, terminating their active sessions and preventing further access to protected pages.

**FR-03 – Role-Based Access Control**

The system shall support two main user roles: Student and Administrator. Students shall have access to their personal profiles, academic records, semester plans, GPA calculations, and academic progress dashboards. Administrators shall have access to academic information management, grading-rule configuration, section management, and administrative reports. The system shall verify user permissions whenever a protected operation is requested and prevent unauthorized users from accessing restricted features, even when attempting to access them through a direct URL.

**FR-04 – Browse Course Catalog**

The system shall provide users with a course catalog displaying available academic courses. Each course entry shall include its course code, title, department, credit hours,

and a brief description. The catalog shall support navigation through large numbers of courses using pagination or an equivalent mechanism. Only active courses shall appear in the default student catalog, while administrators shall be able to view inactive courses for management purposes.

**FR-05 – Search and Filter Courses**

The system shall allow students to search for courses using course codes, course titles, or keywords. It shall also provide filtering options based on department, credit-hour range, and selected academic term when applicable. Users shall be able to combine multiple search and filter conditions to narrow their results. If no courses match the selected conditions, the system shall display an informative message and provide an option to clear the applied filters.

**FR-06 – View Course Details**

The system shall display detailed information about a selected course, including its full description, credit hours, prerequisites, and sections offered during the selected academic term. Each section shall display its identifier, meeting days and times, location or delivery mode, instructor when available, total capacity, and remaining seats when reliable capacity data is available. Missing optional information shall be clearly indicated rather than replaced with assumed values.

**FR-07 – Check Prerequisite Eligibility**

The system shall automatically compare the prerequisites of a selected course with the student's recorded completed courses and applicable grades or academic conditions. It shall identify satisfied and unsatisfied prerequisites and explain which requirements prevent the student from adding a course to a semester plan. If prerequisite information or academic records are missing or incomplete, the system shall flag the eligibility result as requiring confirmation rather than automatically treating the student as eligible.

**FR-08 – Maintain Completed Courses**

The system shall allow students to view their completed courses and associated grades. Depending on the academic-data management policy, students may be permitted to enter or update their academic history, or this information may be maintained by authorized administrators. The system shall prevent duplicate completion records for the same course attempt and validate grades against the configured grading scale. Any successful modification shall update the student's academic progress and GPA calculations accordingly.

**FR-09 – Create Semester Plan**

The system shall allow students to create semester plans for available academic terms and assign each plan a descriptive name or label. Students may maintain multiple draft plans for the same term to compare different course combinations and schedules. The system shall allow students to identify their preferred plan and associate every plan with its owner and selected academic term. Students shall only be able to access and manage their own plans.

**FR-10 – Add Course Section to Plan**

The system shall allow students to add available course sections to their semester plans. Before saving a selection, the system shall validate that the course is offered during the selected term, the section is active, another section of the same course has not already been selected, and the latest available capacity information permits the selection. The system shall also check prerequisite eligibility and clearly flag any unmet requirements. If validation fails, the system shall explain the reason and prevent the invalid selection from being saved.

**FR-11 – Remove or Replace a Planned Section**

The system shall allow students to remove a previously selected section or replace it with another section of the same course. Whenever a section is removed or replaced, the system shall automatically update the plan's total credit hours and weekly timetable. Before accepting a replacement, the system shall validate the alternative section's availability, prerequisites, and meeting times against the other selected sections. A confirmation message shall be displayed after a successful update.

**FR-12 – Detect Schedule Conflicts**

The system shall compare the meeting days and time intervals of all selected sections within a semester plan to identify overlapping classes. It shall detect conflicts occurring on the same day and overlapping time intervals, taking recurring meeting patterns into account. Each detected conflict shall identify the affected sections, conflicting day, and overlapping time interval. Sections with missing or incomplete meeting-time information shall be marked as unverified rather than automatically considered conflict-free.

**FR-13 – Generate Conflict-Free Timetable**

The system shall generate and display a weekly timetable based on the course sections selected by the student. Each class shall appear on its corresponding meeting days and time slots, with different courses visually distinguished for easier identification. If

scheduling conflicts are detected, the system shall highlight the affected sections and indicate that the timetable contains conflicts. The system shall only identify a timetable as conflict-free after verifying all available meeting-time information.

**FR-14 – Suggest Alternative Sections**

When a selected section causes a scheduling conflict or becomes unavailable, the system shall suggest alternative active sections of the same course within the selected academic term. The suggested alternatives shall be checked against the student's other planned sections to identify options that do not introduce additional schedule conflicts. The system shall display relevant meeting times, locations when available, and seat availability. If no suitable alternatives are found, an informative message shall be displayed to the student.

**FR-15 – Save and Reopen Plans**

The system shall allow students to save their semester plans and reopen them during future sessions. Saved plans shall retain their selected courses, sections, academic term, and associated information even after the student logs out. The system shall record the last-updated date and time of each plan. When a saved section becomes inactive or its capacity information changes, the system shall notify the student and flag the affected selection for review when the plan is reopened.

**FR-16 – Calculate Semester GPA**

The system shall calculate an estimated semester GPA using the grades and credit hours associated with completed courses in the selected academic term. The calculation shall follow the configured grading scale and use a credit-weighted formula:

Semester GPA = Sum of (Grade Points × Credit Hours) / Sum of Included Credit Hours.

Courses without assigned grades shall be excluded from the calculation and clearly identified. The system shall handle cases where no completed courses or graded credit hours are available without producing calculation errors. The grading policy used shall be available to the student for reference.

**FR-17 – Calculate Cumulative GPA and Track Academic Progress**

The system shall calculate an estimated cumulative GPA using the student's recorded completed courses and the configured rules for repeated courses and grade inclusion. It shall also summarize the student's completed courses, earned credit hours, remaining credit hours, and unmet prerequisite requirements. The system shall update these values whenever relevant academic records are modified. The calculation shall clearly indicate

the grading scale and academic policies used so that students can understand how their results were obtained.

**FR-18 – Manage Course Catalog (Administrator)**

The system shall allow authorized administrators to create, view, update, deactivate, and, when appropriate, delete course records. Required course information shall include a unique course code, title, department, and credit hours. The system shall validate required fields and prevent duplicate course codes. Before deleting a course, the system shall check whether it is referenced by completed academic records, prerequisite relationships, or active offerings. If historical or academic references exist, the system shall prevent deletion or recommend deactivation to preserve data consistency.

**FR-19 – Manage Offerings, Sections, and Capacity (Administrator)**

The system shall allow administrators to create and update academic-term course offerings and their associated sections. Administrators shall be able to manage section identifiers, meeting days and times, locations or delivery modes, instructor information, active status, and seat capacities. Capacity values shall be validated as non-negative whole numbers. If a capacity reduction affects existing recorded reservations or enrollments, the system shall prevent the change or display an appropriate warning according to the system's capacity-management policy. Students shall be able to view the latest published offering information.

**FR-20 – Manage Prerequisites, Grading Rules, and Administrative Reports**

The system shall allow authorized administrators to create, update, and deactivate prerequisite relationships between courses and configure the grading scale and GPA calculation policies. It shall prevent a course from being assigned itself as a prerequisite and warn administrators about circular prerequisite relationships. Administrators shall also be able to generate summary reports containing course offerings, section capacities, available seats, and student planning demand based on the information stored in the system. Changes to academic rules shall be recorded with the responsible administrator and the date and time of modification.

**FR-21 – Display Visual Academic Dashboard**

The system shall provide students with a visual dashboard presenting an overview of their academic performance and progress. The dashboard shall use charts, graphs, and progress indicators to display information such as completed versus remaining credit hours, completed courses, semester GPA, and cumulative GPA. The displayed information shall be generated using the student's recorded academic data and shall be updated whenever relevant records change. Each visualization shall include clear labels and values to help students understand their academic standing.

**FR-22 – Display Graduation Progress Overview**

The system shall provide students with an estimated overview of their progress toward completing their degree requirements. It shall compare the student's completed courses and earned credit hours against the configured degree requirements, including required courses, electives, and total credit hours where applicable. The system shall identify completed requirements, remaining requirements, and courses that still need to be completed. The results shall be presented using progress indicators and visual summaries to help students plan future semesters. The system shall clearly state that graduation progress is an estimate and does not represent an official graduation decision.

### 3.2 Use Case Model

The use case model identifies the main interactions between the system and its external actors. The Student actor uses planning and academic-progress services. The Administrator actor maintains shared academic data and produces administrative summaries. The diagrams are separated by workflow for readability; a use case appearing in a diagram is described only once in Section 3.2.2.

#### 3.2.1 Use Case Diagram – Student Account and Academic Information

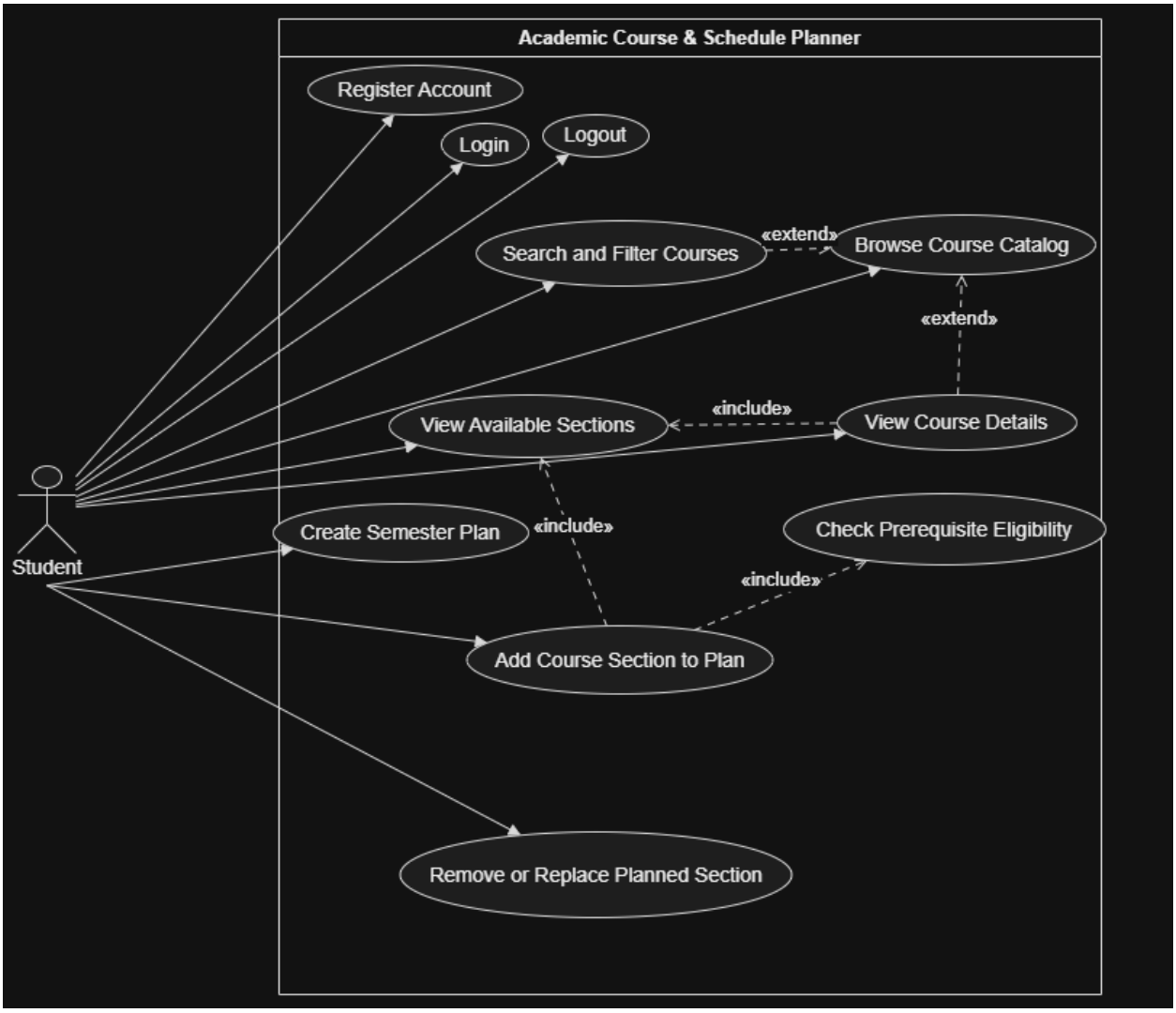



Figure 1. Student account and course-planning use cases. Students can register, log in, browse and search the course catalog, view course details, check prerequisite eligibility, and explore available sections. Adding a section includes prerequisite and section-availability validation, while searching and viewing additional course details extend the browsing process.

### 3.2.1 Figure 2 – Timetable Management and Academic Progress

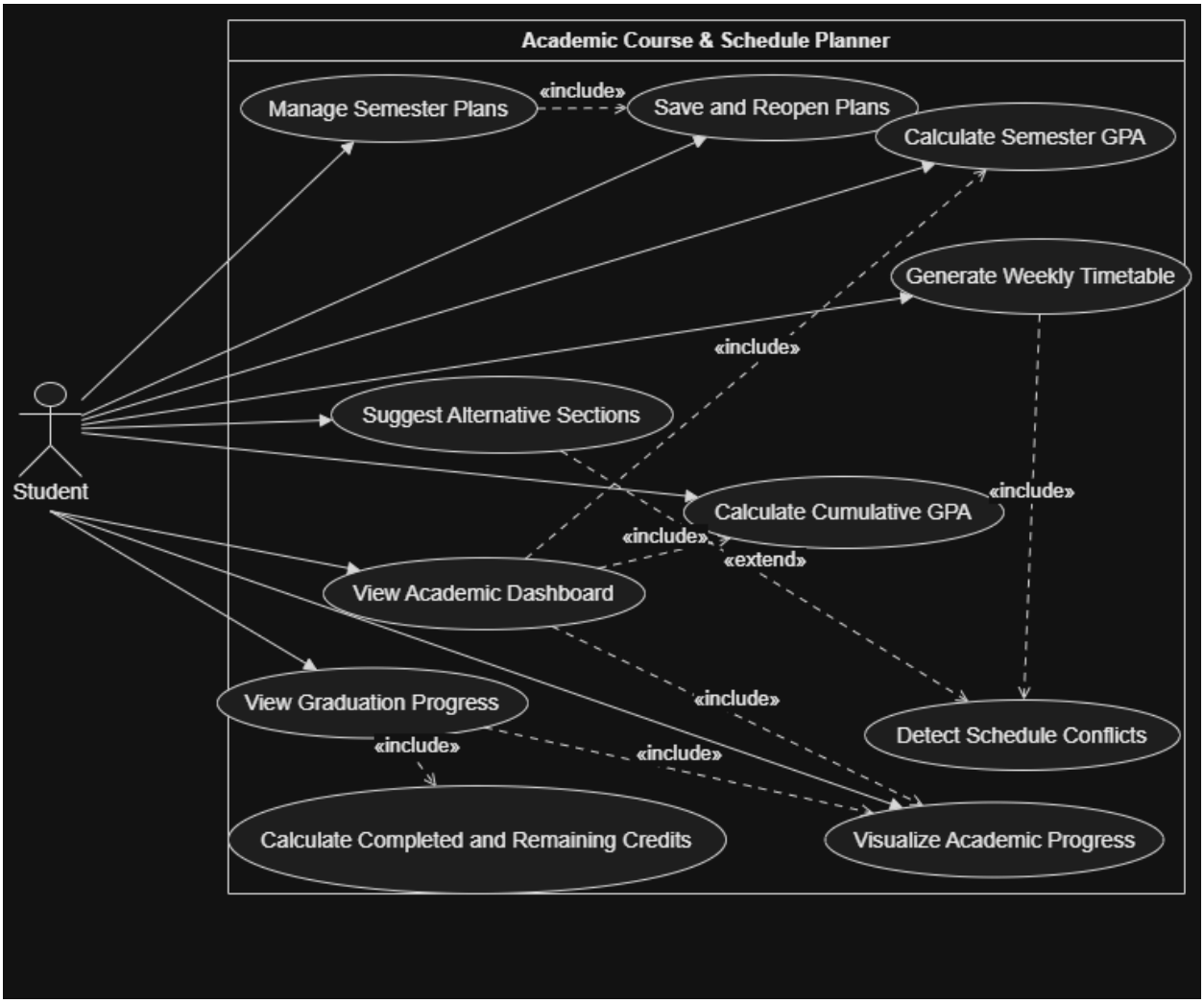



Figure 2. Student timetable management and academic-progress use cases. Students can manage multiple semester plans, and each plan can contain multiple course sections. Generating a weekly timetable includes schedule-conflict detection, while alternative section suggestions extend the process when a conflict occurs. The academic dashboard includes semester GPA, cumulative GPA, and visual academic-progress information. Graduation progress includes calculating completed and remaining credit hours.

### 3.2.1 Figure 3 – Administrative Management

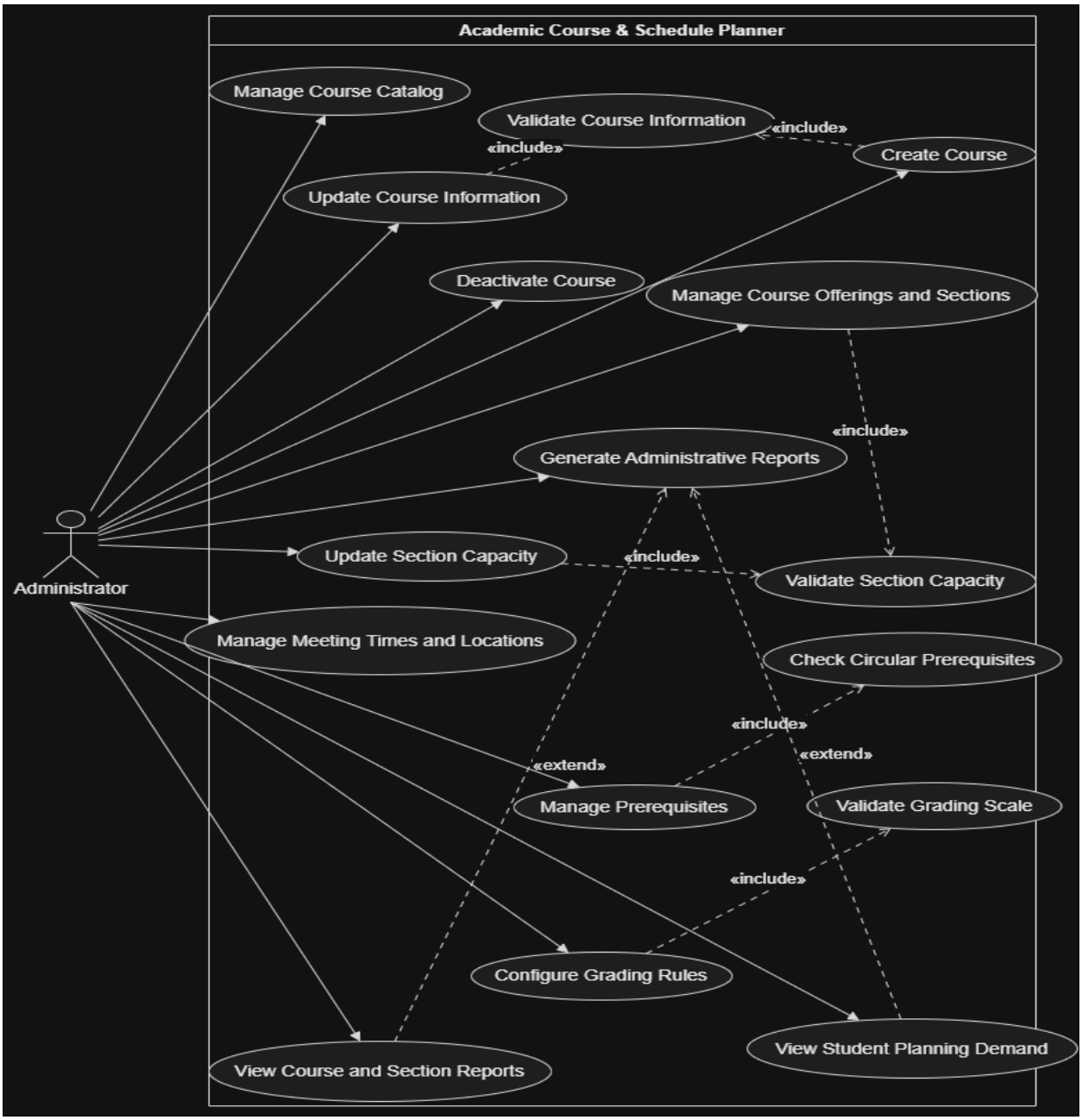



Figure 3. Administrative management use cases. Administrators manage the course catalog, course offerings, sections, prerequisites, grading rules, and administrative reports. Creating or updating course information includes data validation, while managing prerequisites includes checking for circular prerequisite relationships. Managing sections includes capacity validation, and generating administrative reports can be extended to display course and section information or student planning demand.

### 3.2.2 Use Case Descriptions

**Figure 1 – Student Account and Course Planning**

Diagram Description:

This use case diagram illustrates how students interact with the Academic Course & Schedule Planner to access their accounts and explore available academic courses. It shows the main activities involved in browsing the course catalog, searching and filtering courses, viewing course details, checking prerequisite eligibility, and creating semester plans. Students can also view available sections and add, remove, or replace course sections within their plans. The diagram highlights the relationships between these activities, including the use of prerequisite checking when adding a course section to a semester plan.

**UC-01 – Register Account**

| Field | Description |
|---|---|
| **Primary Actor** | **Student** |
| **Description** | **Allows a new student to create an account in the system.** |
| **Preconditions** | **The student does not already have an account.** |
| **Postconditions** | **A new student account is created successfully.** |
| **Main Success Scenario** | **1. The student opens the registration page.**<br>**2. The system displays the registration form.**<br>**3. The student enters their name, university email, and password.**<br>**4. The system validates the entered information.**<br>**5. The system creates the account and displays a confirmation message.** |
| **Alternative / Exception Flows** | **If required information is missing, the system requests correction. If the email is already registered or the password does not meet the configured policy, registration is rejected. If email verification is enabled, the account remains pending until verification.** |

## UC-02 – Login

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Allows a registered student to access their account. |
| Preconditions | The student has an existing account. |
| Postconditions | The student is authenticated and redirected to their dashboard. |
| Main Success Scenario | 1. The student opens the login page.<br>2. The student enters their email and password.<br>3. The system validates the credentials.<br>4. The system creates an authenticated session.<br>5. The student is redirected to their dashboard. |
| Alternative / Exception Flows | If the credentials are incorrect, the system displays an error message and allows the student to try again. |

## UC-03 – Logout

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Allows the student to securely end their current session. |
| Preconditions | The student is logged in. |
| Postconditions | The active session is terminated. |
| Main Success Scenario | 1. The student selects the logout option.<br>2. The system terminates the active session.<br>3. The system redirects the student to the login page. |

| Field | Description |
| --- | --- |
| Alternative / Exception Flows | If the session has already expired, the system redirects the student to the login page. |

**UC-04 – Browse Course Catalog**

| Field | Description |
| --- | --- |
| Primary Actor | Student |
| Description | Allows students to explore the courses available in the academic catalog. |
| Preconditions | The student has access to the system. |
| Postconditions | The available course catalog is displayed. |
| Main Success Scenario | 1. The student opens the course catalog.<br>2. The system retrieves active course records.<br>3. The system displays course codes, titles, departments, credit hours, and brief descriptions.<br>4. The student browses the available courses. |
| Alternative / Exception Flows | If no active courses are available, the system displays an informative message. |

**UC-05 – Search and Filter Courses**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Allows students to find specific courses using search keywords and filtering options. |
| Preconditions | The course catalog is available. |
| Postconditions | Courses matching the search and filter conditions are displayed. |
| Main Success Scenario | 1. The student opens the course catalog.<br>2. The student enters a course code, title, or keyword.<br>3. The student selects optional filters such as department or credit hours.<br>4. The system processes the search conditions.<br>5. The system displays matching courses. |
| Alternative / Exception Flows | If no matching courses are found, the system displays a no-results message. The student may clear the filters and search again. |

**UC-06 – View Course Details**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Allows students to view detailed academic information about a selected course. |
| Preconditions | The selected course exists in the catalog. |
| Postconditions | The course details and available sections are displayed. |
| Main Success Scenario | 1. The student selects a course from the catalog.<br>2. The system retrieves the course information.<br>3. The system displays the description, credit hours, and prerequisites.<br>4. The system displays the sections available for the selected academic term. |
| Alternative / Exception Flows | If the course has no sections available in the selected term, the system displays the course details and indicates that no sections are currently offered. |

**UC-07 – Check Prerequisite Eligibility**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Checks whether a student has completed the required courses to take a selected course. |
| Preconditions | The selected course has prerequisite information, and the student's academic records are available. |
| Postconditions | The prerequisite eligibility result is displayed. |
| Main Success Scenario | 1. The student selects a course.<br>2. The system retrieves its prerequisite requirements.<br>3. The system compares the requirements with the student's completed courses and grades.<br>4. The system identifies satisfied and unsatisfied prerequisites.<br>5. The system displays the eligibility result. |
| Alternative / Exception Flows | If prerequisites or academic records are incomplete, the system marks the result as requiring confirmation. If one or more prerequisites are not satisfied, the system explains which requirements remain unmet. |

**UC-08 – View Available Sections**

| Field | Description |
|---|---|
| **Primary Actor** | **Student** |
| **Description** | **Allows students to view the sections offered for a particular course during an academic term.** |
| **Preconditions** | **The selected course exists in the catalog.** |
| **Postconditions** | **Available section information is displayed.** |
| **Main Success Scenario** | **1. The student selects a course.**<br>**2. The system retrieves its active sections for the selected term.**<br>**3. The system displays section identifiers, meeting times, locations, instructors, and capacity information when available.** |
| **Alternative / Exception Flows** | **If no active sections are available, the system informs the student that the course is not currently offered.** |

**UC-09 – Create Semester Plan**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Allows students to create a new academic semester plan. |
| Preconditions | The student is logged in and an academic term is available. |
| Postconditions | A new semester plan is created and associated with the student's account. |
| Main Success Scenario | 1. The student opens the semester planning page.<br>2. The student selects an academic term.<br>3. The student enters a name or label for the plan.<br>4. The system creates the plan.<br>5. The new plan appears in the student's saved plans. |
| Alternative / Exception Flows | If required information is missing, the system requests correction. |

**UC-10 – Add Course Section to Plan**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Allows students to add a selected course section to an existing semester plan. |
| Preconditions | The student is logged in, a semester plan exists, and the selected section is active. |
| Postconditions | The selected section is added to the semester plan if all required validations pass. |
| Main Success Scenario | 1. The student opens a semester plan.<br>2. The student selects a course and an available section.<br>3. The system checks prerequisite eligibility.<br>4. The system validates section availability, capacity, and duplicate course selections.<br>5. The system adds the section to the plan.<br>6. The system confirms the successful update. |
| Alternative / Exception Flows | If prerequisites are not satisfied, the system displays the unmet requirements. If the section is inactive, full, or already represented in the plan, the system rejects the selection and explains the reason. |

**UC-11 – Remove or Replace Planned Section**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Allows students to remove a selected section or replace it with another section of the same course. |
| Preconditions | The student has an existing semester plan containing at least one selected section. |
| Postconditions | The semester plan is updated and its timetable and credit-hour total are recalculated. |
| Main Success Scenario | 1. The student opens a semester plan.<br>2. The student selects a section to remove or replace.<br>3. The system displays the available action.<br>4. The student confirms the action or selects an alternative section.<br>5. The system validates the change.<br>6. The system updates the plan and displays a confirmation. |
| Alternative / Exception Flows | If the replacement section is unavailable or invalid, the system rejects the change and keeps the previous selection unchanged. |

**UC-12 – Manage Semester Plans**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Allows students to manage their existing semester plans. |
| Preconditions | The student is logged in. |
| Postconditions | The selected semester plan is created, viewed, or modified as requested. |
| Main Success Scenario | 1. The student opens the semester planning page.<br>2. The system displays the student's saved plans.<br>3. The student selects a plan.<br>4. The system displays its courses, sections, and total credit hours.<br>5. The student makes the required changes. |
| Alternative / Exception Flows | If the student has no saved plans, the system displays an option to create a new plan. |

**UC-13 – Save and Reopen Plans**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Allows students to save their semester plans and access them during future sessions. |
| Preconditions | The student is logged in and has an existing semester plan. |
| Postconditions | The plan is saved and can be retrieved in a later session. |
| Main Success Scenario | 1. The student opens a semester plan.<br>2. The student selects the save option.<br>3. The system stores the selected courses and sections.<br>4. The system records the last-updated date and time.<br>5. The student can reopen the plan later. |
| Alternative / Exception Flows | If a selected section becomes inactive or its information changes, the system flags the affected section and asks the student to review the plan. |

**UC-14 – Generate Weekly Timetable**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Generates a visual weekly timetable based on the sections selected in a semester plan. |
| Preconditions | The student has a semester plan containing at least one selected section. |
| Postconditions | A weekly timetable is generated and displayed. |
| Main Success Scenario | 1. The student opens a semester plan.<br>2. The student selects the timetable view.<br>3. The system retrieves the meeting days and times of the selected sections.<br>4. The system organizes the sessions into a weekly timetable.<br>5. The system displays the timetable and highlights any detected conflicts. |
| Alternative / Exception Flows | If meeting-time information is incomplete, the system marks the affected sections as unverified. If the plan contains overlapping sessions, the system highlights the conflicts. |

**UC-15 – Detect Schedule Conflicts**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Identifies overlapping class sessions within a student's semester plan. |
| Preconditions | The semester plan contains selected sections with meeting-time information. |
| Postconditions | The system displays the detected conflicts or confirms that no conflicts were found in verified meeting times. |
| Main Success Scenario | 1. The system retrieves all selected sections.<br>2. It compares their meeting days and time intervals.<br>3. It identifies overlapping sessions.<br>4. It displays the affected courses and conflicting time intervals. |
| Alternative / Exception Flows | If meeting-time information is missing, the system marks the relevant section as unverified rather than considering it conflict-free. If no conflicts exist, the system displays a confirmation. |

**UC-16 – Suggest Alternative Sections**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Suggests alternative course sections when a selected section causes a scheduling conflict or becomes unavailable. |
| Preconditions | The student has a semester plan containing a conflicting or unavailable section. |
| Postconditions | Suitable alternative sections are displayed, if available. |
| Main Success Scenario | 1. The system identifies a conflicting or unavailable section.<br>2. The system retrieves other active sections of the same course.<br>3. It compares their meeting times with the student's existing plan.<br>4. It filters out sections that introduce additional conflicts.<br>5. It displays suitable alternatives with their meeting times and availability information. |
| Alternative / Exception Flows | If no suitable alternatives exist, the system informs the student that no conflict-free alternative was found. |

**UC-17 – Calculate Semester GPA**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Calculates the student's estimated GPA for a selected academic semester. |
| Preconditions | The student's completed courses, grades, and credit hours for the selected term are available. |
| Postconditions | The semester GPA is calculated and displayed. |
| Main Success Scenario | 1. The student opens the GPA section.<br>2. The student selects an academic term.<br>3. The system retrieves completed courses and their grades and credit hours.<br>4. The system converts grades into grade points using the configured grading scale.<br>5. The system calculates the credit-weighted GPA.<br>6. The result is displayed. |
| Alternative / Exception Flows | Courses without assigned grades are excluded. If no graded courses are available, the system displays a message indicating that the GPA cannot yet be calculated. |

**UC-18 – Calculate Cumulative GPA**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Calculates the student's estimated cumulative GPA using academic records from completed semesters. |
| Preconditions | The student's completed academic records and applicable grading rules are available. |
| Postconditions | The cumulative GPA is calculated and displayed. |
| Main Success Scenario | 1. The student opens the cumulative GPA section.<br>2. The system retrieves the student's completed courses and grades.<br>3. The system applies the configured rules for repeated courses and grade inclusion. |

| Field | Description |
|---|---|
| | **4. The system calculates the cumulative GPA.**<br>**5. The result and grading policy are displayed.** |
| **Alternative / Exception Flows** | **If academic records are missing or incomplete, the system displays a warning that the result may be inaccurate. If no graded courses are available, the system displays an appropriate message.** |

**UC-19 – View Academic Dashboard**

| Field | Description |
|---|---|
| **Primary Actor** | **Student** |
| **Description** | **Provides students with a visual summary of their academic performance and progress.** |
| **Preconditions** | **The student is logged in.** |
| **Postconditions** | **The academic dashboard is displayed with the available academic information.** |
| **Main Success Scenario** | **1. The student opens the dashboard.**<br>**2. The system retrieves the student's academic records and semester information.**<br>**3. The system prepares the relevant academic statistics.**<br>**4. The system displays GPA information, completed courses, earned credit hours, and other academic indicators through visual elements.** |
| **Alternative / Exception Flows** | **If academic data is incomplete, the system displays the available information and indicates which statistics cannot yet be calculated.** |

**UC-20 – Visualize Academic Progress**

| Field | Description |
|---|---|
| **Primary Actor** | **Student** |
| **Description** | **Displays academic progress through charts, graphs, and progress indicators.** |
| **Preconditions** | **The student has recorded academic information.** |

| Field | Description |
|---|---|
| Postconditions | Visual representations of the student's academic progress are displayed. |
| Main Success Scenario | 1. The student opens the academic progress section.<br>2. The system retrieves completed courses and earned credit hours.<br>3. The system calculates completed and remaining credit hours based on configured degree requirements.<br>4. The system generates visual charts and progress indicators.<br>5. The student reviews the displayed information. |
| Alternative / Exception Flows | If degree requirements or academic records are unavailable, the system displays a message explaining that the progress visualization is incomplete. |

**UC-21 – View Graduation Progress**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Provides an estimated overview of the student's progress toward completing degree requirements. |
| Preconditions | The student's academic records and configured degree requirements are available. |
| Postconditions | The student's completed and remaining graduation requirements are displayed. |
| Main Success Scenario | 1. The student opens the graduation progress section.<br>2. The system retrieves the student's completed courses and degree requirements.<br>3. The system compares completed courses with required courses, electives, and credit-hour requirements.<br>4. The system identifies completed and remaining requirements.<br>5. The system displays the results using progress indicators and visual summaries. |
| Alternative / Exception Flows | If degree requirements or academic records are incomplete, the system informs the student that the graduation progress result is only partially available. |

**UC-22 – Calculate Completed and Remaining Credits**

| Field | Description |
|---|---|
| Primary Actor | Student |
| Description | Calculates the total earned credit hours and the remaining credit hours required for degree completion. |
| Preconditions | The student's completed course records and total degree credit-hour requirements are available. |
| Postconditions | Completed and remaining credit hours are calculated and displayed. |
| Main Success Scenario | 1. The system retrieves the student's completed courses.<br>2. It calculates the total earned credit hours.<br>3. It retrieves the required credit hours for the student's degree.<br>4. It calculates the remaining credit hours.<br>5. It displays both values to the student. |
| Alternative / Exception Flows | If the degree's total credit-hour requirement is unavailable, the system displays the earned credit hours but cannot calculate the remaining amount. |

**UC-23 – Manage Course Catalog**

| Field | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Allows administrators to manage the academic course catalog. |
| Preconditions | The administrator is logged in and has the required permissions. |
| Postconditions | Course records are created, updated, deactivated, or deleted according to the requested operation. |
| Main Success Scenario | 1. The administrator opens the course management section.<br>2. The system displays existing course records. |

| Field | Description |
|---|---|
| | **3. The administrator selects an operation.**<br>**4. The administrator enters or modifies the required course information.**<br>**5. The system validates the information.**<br>**6. The system saves the changes.** |
| **Alternative / Exception Flows** | **If required information is missing or a duplicate course code is entered, the system rejects the operation. If a course is referenced by academic records or prerequisites, the system prevents unsafe deletion and recommends deactivation.** |

**UC-24 – Create Course**

| Field | Description |
|---|---|
| **Primary Actor** | **Administrator** |
| **Description** | **Allows administrators to add a new course to the academic catalog.** |
| **Preconditions** | **The administrator is logged in and has permission to manage courses.** |
| **Postconditions** | **A new course record is added to the catalog.** |
| **Main Success Scenario** | **1. The administrator selects the option to create a course.**<br>**2. The system displays a course information form.**<br>**3. The administrator enters the course code, title, department, credit hours, and description.**<br>**4. The system validates the entered information.**<br>**5. The system saves the new course.** |
| **Alternative / Exception Flows** | **If required fields are missing or the course code already exists, the system displays an error and requests correction.** |

**UC-25 – Update Course Information**

| Field | Description |
|---|---|
| **Primary Actor** | **Administrator** |

| Field | Description |
|---|---|
| Description | Allows administrators to modify existing course information. |
| Preconditions | The selected course exists and the administrator has the required permissions. |
| Postconditions | The selected course record is updated. |
| Main Success Scenario | 1. The administrator searches for a course.<br>2. The administrator selects the course to update.<br>3. The system displays its current information.<br>4. The administrator modifies the required fields.<br>5. The system validates and saves the changes. |
| Alternative / Exception Flows | If the entered information is invalid, the system rejects the update and displays the required corrections. |

**UC-26 – Deactivate Course**

| Field | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Allows administrators to deactivate a course that should no longer appear as an active offering in the catalog. |
| Preconditions | The selected course exists and the administrator has permission to manage courses. |
| Postconditions | The course is marked as inactive and is no longer displayed in the default student catalog. |
| Main Success Scenario | 1. The administrator searches for a course.<br>2. The administrator selects the deactivate option.<br>3. The system requests confirmation.<br>4. The administrator confirms the action.<br>5. The system updates the course status. |

| Field | Description |
|---|---|
| Alternative / Exception Flows | If the administrator cancels the operation, the course remains active. Historical records associated with the course are preserved. |

**UC-27 – Validate Course Information**

| Field | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Checks course information before creating or updating a course record. |
| Preconditions | The administrator has entered or modified course information. |
| Postconditions | The system accepts valid information or displays validation errors. |
| Main Success Scenario | 1. The administrator submits the course form.<br>2. The system checks required fields.<br>3. The system verifies that the course code is unique.<br>4. The system validates credit hours and other required information.<br>5. The system confirms that the information is valid. |
| Alternative / Exception Flows | If any field is missing, duplicated, or invalid, the system identifies the problem and requests correction. |

**UC-28 – Manage Course Offerings and Sections**

| Field | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Allows administrators to create and maintain course offerings and their associated sections for different academic terms. |
| Preconditions | The administrator is logged in and the relevant course exists. |
| Postconditions | Course offerings and section records are created or updated. |

| Field | Description |
|---|---|
| Main Success Scenario | 1. The administrator opens the offerings management section.<br>2. The system displays existing offerings and sections.<br>3. The administrator selects a course and academic term.<br>4. The administrator creates or modifies section information.<br>5. The system validates and saves the changes. |
| Alternative / Exception Flows | If the selected course does not exist or required section information is invalid, the system rejects the operation and displays an error. |

**UC-29 – Update Section Capacity**

| Field | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Allows administrators to modify the maximum seat capacity of a course section. |
| Preconditions | The section exists and the administrator has the required permissions. |
| Postconditions | The section capacity is updated if the new value passes validation. |
| Main Success Scenario | 1. The administrator selects a course section.<br>2. The system displays its current capacity.<br>3. The administrator enters a new capacity value.<br>4. The system validates the value.<br>5. The system saves the updated capacity. |
| Alternative / Exception Flows | Negative or non-integer values are rejected. If the new capacity affects existing reservations or enrollments, the system prevents the change or displays a warning according to the capacity-management policy. |

**UC-30 – Manage Meeting Times and Locations**

| Field | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Allows administrators to maintain the meeting days, times, locations, and delivery modes of course sections. |
| Preconditions | The selected section exists. |
| Postconditions | The section's meeting information is updated. |
| Main Success Scenario | 1. The administrator selects a section.<br>2. The system displays its current meeting information.<br>3. The administrator enters or modifies meeting days, start and end times, location, or delivery mode.<br>4. The system validates the information.<br>5. The system saves the changes. |
| Alternative / Exception Flows | If meeting times are invalid or required information is missing, the system displays an error and requests correction. |

**UC-31 – Validate Section Capacity**

| Field | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Validates the capacity information entered for a course section. |
| Preconditions | The administrator is creating or updating a section capacity. |
| Postconditions | The capacity is accepted or rejected based on the configured rules. |
| Main Success Scenario | 1. The administrator enters a capacity value.<br>2. The system checks that the value is a non-negative whole number.<br>3. The system checks existing enrollment or reservation records if applicable. |

| Field | Description |
|---|---|
| | 4. The system confirms whether the new capacity is valid. |
| Alternative / Exception Flows | If the capacity is negative, not a whole number, or lower than existing reservations where applicable, the system rejects the change or displays the appropriate warning. |

**UC-32 – Manage Prerequisites**

| Field | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Allows administrators to create, modify, and deactivate prerequisite relationships between courses. |
| Preconditions | The administrator is logged in and the relevant courses exist in the catalog. |
| Postconditions | The prerequisite relationships are updated in the academic database. |
| Main Success Scenario | 1. The administrator opens prerequisite management.<br>2. The administrator selects a course.<br>3. The administrator adds, modifies, or removes a prerequisite relationship.<br>4. The system validates the relationship.<br>5. The system saves the changes. |
| Alternative / Exception Flows | The system rejects a course being assigned itself as a prerequisite and warns about circular prerequisite relationships. |

**UC-33 – Check Circular Prerequisites**

| Field | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Checks prerequisite relationships to identify invalid circular dependencies between courses. |
| Preconditions | The administrator is adding or modifying prerequisite relationships. |

| Field | Description |
|---|---|
| Postconditions | The system identifies whether the prerequisite structure contains circular dependencies. |
| Main Success Scenario | 1. The administrator submits a prerequisite change.<br>2. The system retrieves the relevant prerequisite relationships.<br>3. The system checks for circular dependencies.<br>4. The system displays the validation result. |
| Alternative / Exception Flows | If a circular relationship is detected, the system warns the administrator and prevents saving the invalid relationship. |

**UC-34 – Configure Grading Rules**

| Field | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Allows administrators to configure the grading scale and GPA calculation policies used by the system. |
| Preconditions | The administrator is authenticated and has permission to manage academic rules. |
| Postconditions | The configured grading rules are saved and made available for GPA calculations. |
| Main Success Scenario | 1. The administrator opens grading-rule management.<br>2. The system displays the current grading scale.<br>3. The administrator enters or modifies grade values and corresponding grade points.<br>4. The administrator configures applicable GPA policies.<br>5. The system validates and saves the changes. |
| Alternative / Exception Flows | If grading values are missing, duplicated, or invalid, the system rejects the changes and requests correction. |

**UC-35 – Validate Grading Scale**

| Field | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Validates the grading scale before it is saved and used in GPA calculations. |
| Preconditions | The administrator has entered or modified grading-scale information. |
| Postconditions | The grading scale is accepted or rejected based on validation results. |
| Main Success Scenario | 1. The administrator submits the grading scale.<br>2. The system checks that all required grade values are present.<br>3. The system checks that the corresponding grade points are valid.<br>4. The system verifies that there are no conflicting or duplicate grade definitions.<br>5. The system confirms whether the grading scale can be saved. |
| Alternative / Exception Flows | If the grading scale contains invalid or conflicting values, the system displays the errors and requests correction. |

**UC-36 – Generate Administrative Reports**

| Field | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Allows administrators to generate reports summarizing academic course offerings, section capacities, and student planning demand. |
| Preconditions | The administrator is logged in and relevant academic data is available. |
| Postconditions | The requested administrative report is generated and displayed. |
| Main Success Scenario | 1. The administrator opens the reporting section.<br>2. The system displays available report categories.<br>3. The administrator selects a report type.<br>4. The system retrieves the relevant academic information.<br>5. The system processes and displays the report. |

| Field | Description |
| --- | --- |
| Alternative / Exception Flows | If the selected report has insufficient data, the system displays an informative message. |

**UC-37 – View Course and Section Reports**

| Field | Description |
| --- | --- |
| Primary Actor | Administrator |
| Description | Allows administrators to view summarized information about courses, offerings, section capacities, and available seats. |
| Preconditions | The administrator is authenticated and course and section information is available. |
| Postconditions | The selected course and section report is displayed. |
| Main Success Scenario | 1. The administrator opens the administrative reports section.<br>2. The administrator selects the course and section report.<br>3. The system retrieves course offerings and section information.<br>4. The system summarizes section capacities and availability.<br>5. The system displays the report. |
| Alternative / Exception Flows | If no relevant course or section data is available, the system displays a message indicating that no report data was found. |

**UC-38 – View Student Planning Demand**

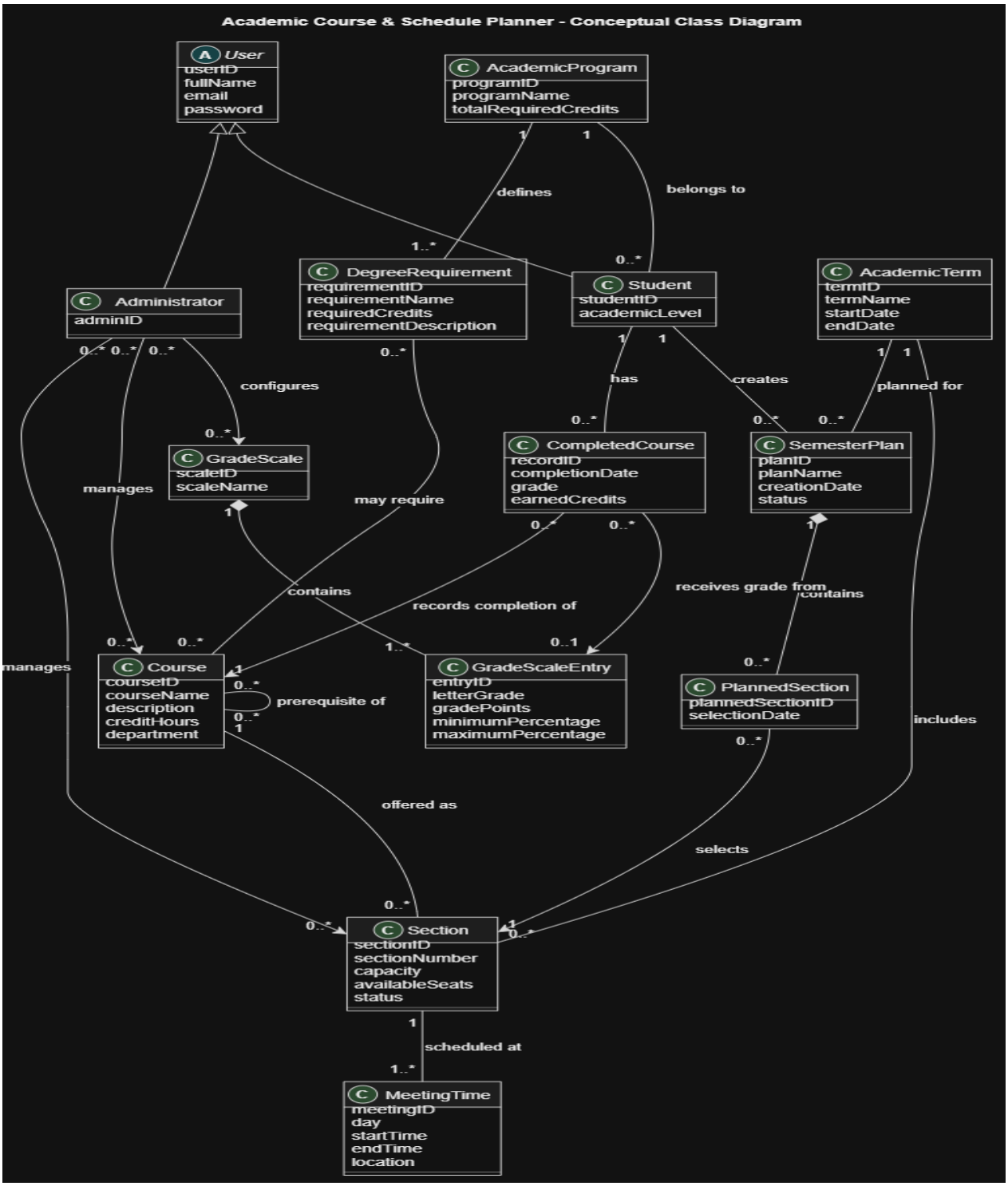
| Field | Description |
|---|---|
| Primary Actor | Administrator |
| Description | Allows administrators to view summaries of student demand for courses and sections based on the semester plans stored in the system. |
| Preconditions | The administrator is authenticated and student planning data is available. |
| Postconditions | A summary of student planning demand is displayed. |
| Main Success Scenario | 1. The administrator opens the administrative reports section.<br>2. The administrator selects the student planning demand report.<br>3. The system retrieves saved student semester plans.<br>4. The system summarizes the number of plans containing each course or section.<br>5. The system displays the results. |
| Alternative / Exception Flows | If no student plans contain relevant selections, the system displays an informative message indicating that no planning-demand data is available. |

## 3.3 Domain Model

The domain model describes the main concepts of the Academic Course & Schedule Planner and how they relate to each other. It focuses on the academic information managed by the system, including students, courses, sections, semester plans, completed courses, prerequisites, and academic progress.

### 3.3.1 Conceptual Class Diagram

The following UML class diagram represents the main classes, their important attributes, and the relationships between them. It also includes multiplicities to show how many objects can participate in each relationship.



### 3.3.2 Class Descriptions

The following table explains the purpose of each class, its main attributes, and its relationships with other classes. Attributes are listed without data types, as required for a conceptual class diagram.

**A. User and Academic Information Classes**

| Class | Attributes | Associations | Purpose |
|---|---|---|---|
| **User** | userID, fullName, email, password | General parent class of Student and Administrator. | Represents a person who can access the system. It contains the common account information shared by all users. |
| **Student** | studentID, academicLevel | Inherits from User. Belongs to one AcademicProgram. Creates multiple SemesterPlans and has multiple CompletedCourses. | Represents a student who uses the application to plan semesters, check academic progress, and calculate GPA estimates. |
| **Administrator** | adminID | Inherits from User. Manages Courses, Sections, and GradeScales. | Represents an authorized user responsible for maintaining academic information and system settings. |
| **AcademicProgram** | programID, programName, totalRequiredCredits | Has multiple Students and defines multiple DegreeRequirements. | Represents an academic degree program and its overall credit-hour requirements. |

**B. Course and Schedule Classes**

| Class | Attributes | Associations | Purpose |
|---|---|---|---|
| **Course** | courseID, courseName, description, creditHours, department | Can have multiple prerequisite Courses. Has multiple Sections and can be linked to multiple DegreeRequirements. | Represents a university course available in the academic catalog. |
| **Section** | sectionID, sectionNumber, capacity, availableSeats, status | Belongs to one Course and one AcademicTerm. Has one or more MeetingTimes. Can be selected in multiple PlannedSections. | Represents a specific offering of a course during an academic term, including its capacity and schedule. |
| **AcademicTerm** | termID, termName, startDate, endDate | Includes multiple Sections and is associated with multiple SemesterPlans. | Represents an academic semester or term in which courses and sections are offered. |
| **MeetingTime** | meetingID, day, startTime, endTime, location | Belongs to one Section. | Stores the days, times, and locations at which a section meets. This information is used to detect timetable conflicts. |

**C. Semester Planning Classes**

| Class | Attributes | Associations | Purpose |
|---|---|---|---|
| **SemesterPlan** | planID, planName, creationDate, status | Belongs to one Student and one AcademicTerm. Contains multiple PlannedSections. | Represents a student's saved academic plan for a particular semester. |
| **PlannedSection** | plannedSectionID, selectionDate | Belongs to one SemesterPlan and references one Section. | Represents the selection of a particular section within a semester plan. It allows the system to manage and modify selected sections. |

**D. Academic Records and GPA Classes**

| Class | Attributes | Associations | Purpose |
|---|---|---|---|
| **CompletedCourse** | recordID, completionDate, grade, earnedCredits | Belongs to one Student and references one Course. May be associated with one GradeScaleEntry. | Stores information about a course that a student has completed, including the grade and earned credit hours. |
| **GradeScale** | scaleID, scaleName | Contains multiple GradeScaleEntries and can be configured by Administrators. | Represents the grading system used to calculate semester and cumulative GPA. |
| **GradeScaleEntry** | entryID, letterGrade, gradePoints, minimumPercentage, maximumPercentage | Belongs to one GradeScale and may be associated with multiple CompletedCourses. | Defines the relationship between a letter grade, its grade points, and its corresponding percentage range. |

**E. Graduation Requirements Class**

| Class | Attributes | Associations | Purpose |
|---|---|---|---|
| **DegreeRequirement** | requirementID, requirementName, requiredCredits, requirementDescription | Belongs to one AcademicProgram and can be associated with multiple Courses. | Represents a graduation requirement used to estimate a student's academic progress and remaining credit hours. |

The GPA calculations and visual academic dashboard are system functions rather than separate academic entities in this conceptual model. The system uses information from CompletedCourse, GradeScale, SemesterPlan, and DegreeRequirement to calculate GPA estimates, completed credit hours, remaining credit hours, and graduation progress.

**3.4 Non-Functional Requirements**

Non-functional requirements describe the quality attributes and performance expectations of the Academic Course & Schedule Planner. They specify how well the system should operate rather than describing individual functions.

Each requirement below includes an explanation, a proposed target, and a method for testing whether the requirement has been achieved.

**NFR-01: Performance**

Description:

The system should respond quickly to user actions, including browsing the course catalog, searching for courses, checking prerequisites, generating timetables, and calculating GPA. Normal user operations should not take an excessive amount of time.

Testing Method:

The development team will perform performance testing by measuring the response time of common operations under normal usage conditions.

Success Criteria:

- At least 95% of normal user requests should receive a response within 3 seconds.
- Timetable generation and conflict detection should complete within 5 seconds for a typical semester plan.

**NFR-02: Usability**

Description:

The system should provide a simple, clear, and user-friendly interface. Students should be able to navigate between the course catalog, semester plans, timetable, GPA calculator, and academic dashboard without requiring advanced technical knowledge.

Testing Method:

Usability testing will be conducted with a group of students. They will be asked to complete common tasks, such as creating a semester plan, adding a course section, and viewing their academic progress.

Success Criteria:

- At least 80% of test participants should successfully complete the assigned tasks without assistance.
- Users should be able to understand the main navigation and basic system functions with minimal guidance.

**NFR-03: Reliability**

Description:

The system should operate consistently and produce accurate results when checking schedule conflicts, validating prerequisites, and calculating GPA. It should also prevent saved academic plans from being lost during normal system operation.

Testing Method:

The team will perform functional and reliability testing by repeating important operations with different input values and checking whether the results are consistent. Saved plans will also be reopened after saving and after restarting the application.

Success Criteria:

- All tested schedule-conflict cases should produce the expected results.
- GPA calculations should match independently calculated expected results for the same grading rules.
- Saved semester plans should remain available after users log out and log in again.

**NFR-04: Security**

Description:

The system should protect user accounts and academic information from unauthorized access. Users must authenticate before accessing protected features, and students should only be able to access their own academic plans and records. Administrative functions should only be available to authorized administrators.

Testing Method:

The development team will perform security testing by attempting to access protected pages without logging in, accessing another student's records, and using student accounts to access administrative functions.

Success Criteria:

- Unauthorized users must not be able to access protected system features.
- Students must not be able to view or modify another student's private academic records.
- Only authorized administrators should be able to modify course information, section offerings, and grading rules.
- Passwords must be stored securely rather than as plain text.

**NFR-05: Compatibility**

Description:

The system should work correctly on commonly used web browsers and different device sizes. Its interface should remain usable on desktop computers, laptops, tablets, and smartphones.

Testing Method:

Compatibility testing will be performed using Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari. The application will also be tested at different screen sizes.

Success Criteria:

- All main system functions should work correctly on the supported browsers.
- The interface should adapt to different screen sizes without losing important functionality.
- Users should be able to access the main features on both desktop and mobile devices.

**NFR-06: Maintainability**

Description:

The system should be organized into understandable and manageable components so that developers can identify errors, update existing features, and add new functionality without unnecessarily affecting other parts of the application.

Testing Method:

The development team will review the source code structure and perform a maintenance test by modifying a small feature, such as a grading rule or course attribute, and checking whether unrelated features continue to work correctly.

Success Criteria:

- The source code should be organized into understandable modules.
- Important functions and complex logic should include appropriate comments or documentation.
- Changes to one feature should not cause failures in unrelated features during regression testing.

**NFR-07: Data Integrity**

Description:

The system should maintain accurate and consistent academic information. It should validate important inputs, prevent invalid relationships between academic records, and ensure that GPA and progress calculations use valid data.

Testing Method:

The team will perform validation testing using incomplete, incorrect, duplicate, and inconsistent information. Examples include invalid course credit hours, incorrect grading values, and sections with invalid meeting times.

Success Criteria:

- Invalid academic information should be rejected or flagged with a clear error message.
- A planned section must reference an existing section in the system.
- GPA and academic progress calculations should use the correct stored academic records and configured grading rules.
- Updates to academic information should not create inconsistent records.

**NFR-08: Availability and Recovery**

Description:

The system should be accessible whenever the application server is running and should provide a reasonable recovery process in case of unexpected failures. Users should receive clear feedback if the application becomes temporarily unavailable.

Testing Method:

The team will test the application by restarting the server, simulating unexpected interruptions, and checking whether stored academic plans and records remain available after recovery.

Success Criteria:

- The system should recover and become accessible again after a normal server restart.
- Previously saved academic plans and records should not be lost after a server restart.
- If a temporary failure occurs, the system should display an understandable error message rather than an unexplained failure.

---

## 3.5 External Interface Requirements

This section describes how the Academic Course & Schedule Planner will interact with its users, hardware devices, external software, and communication networks. These requirements define the interfaces needed for users to access the system and for the application to exchange and manage academic information.

### 3.5.1 User Interface

The system will provide a simple, responsive, and user-friendly web interface that allows students and administrators to access the available features easily.

The main user interface components will include:

1. Login and Registration Pages: Allow users to create accounts, log in, and securely access the system using their credentials.

2. Student Dashboard: Display an overview of the student's academic information, including current semester plans, GPA estimates, completed credit hours, remaining credit hours, and academic progress.

3. Course Catalog Page: Allow students to browse, search, and filter available courses by course name, department, credit hours, and other relevant criteria.

4. Course Details Page: Display detailed information about each course, including its description, credit hours, prerequisites, and available sections.

5. Semester Planning Page: Allow students to create and manage semester plans, add or remove course sections, and review their selected courses.

6. Weekly Timetable Page: Display the selected sections in a weekly schedule format. Schedule conflicts should be clearly highlighted to help students identify overlapping classes.

7. GPA Calculator: Display estimated semester and cumulative GPA based on the student's completed courses, grades, and configured grading rules.

8. Academic Progress Dashboard: Present academic information using charts, graphs, and progress indicators. It will help students visualize completed and remaining credit hours, GPA trends, and estimated progress toward graduation.

9. Administrator Dashboard: Allow administrators to manage courses, section offerings, capacities, prerequisites, grading rules, and academic reports.

10. Notifications and Error Messages: Display clear messages when users perform invalid actions, select conflicting sections, fail prerequisite checks, or successfully save their plans.

The interface should adapt to different screen sizes and support modern web browsers, including Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari.

### 3.5.2 Hardware Interface

The Academic Course & Schedule Planner does not require direct interaction with specialized hardware devices such as scanners, sensors, or printers to perform its main functions.

The system will use the following hardware:

- Client Devices: Desktop computers, laptops, tablets, and smartphones will allow students and administrators to access the application through a web browser.
- Server Hardware: A server or cloud-hosted environment will run the application and handle user requests, academic calculations, and database operations.
- Network Devices: Routers, modems, and network equipment will provide internet connectivity between client devices and the application server.

The system will not require additional hardware components. Any device with a supported web browser and a stable internet connection should be sufficient to access the application.

### 3.5.3 Software Interface

The system will interact with software components required for application operation, data storage, and academic information management.

The expected software interfaces include:

1. Web Browser: The application will be accessed through modern web browsers such as Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari. The browser will display the user interface and communicate with the application server.

2. Database Management System: A relational database management system will store and manage user accounts, courses, sections, prerequisites, semester plans, completed courses, and grading information. The application will communicate with the database to retrieve, insert, update, and delete records.

3. Application Server: The application server will process requests from users, execute business logic, validate academic information, detect schedule conflicts, and perform GPA and academic progress calculations.

4. Operating System: The server-side application will run on a supported operating system, while client devices may use Windows, macOS, Linux, Android, or iOS.

5. External Email Service (Optional): If email verification or password recovery is implemented, the system may communicate with an external email service to send verification messages and password reset links.

6. Official University Systems (Future Integration): The initial version will not depend on direct integration with the university's official student information system or registration platform. However, future versions may integrate with these systems through APIs to retrieve verified academic records, course offerings, or registration information, subject to university approval.

### 3.5.4 Communication Interface

The Academic Course & Schedule Planner will use internet-based communication to connect client devices with the application server.

The expected communication requirements include:

1. HTTPS Protocol: The system will use HTTPS (Hypertext Transfer Protocol Secure) to exchange information between users' browsers and the application server. HTTPS will help protect login credentials and academic information during transmission through encryption.

2. HTTP Protocol: HTTP may be used for local development and testing. In the deployed application, HTTPS should be used for all user-facing communication.

3. Client-Server Communication: The browser will send requests to the application server when users log in, search for courses, create semester plans, calculate GPA, or view academic progress. The server will process these requests and return the required results.

4. Database Communication: The application server will communicate with the database through the appropriate database connection interface and protocol. Database access should not be exposed directly to client devices.

5. Data Exchange Format: If the application uses a REST API, JSON (JavaScript Object Notation) will be used to exchange structured information between the frontend and backend.

6. Network Requirements: Users will need a stable internet connection to access the application, retrieve academic information, and save changes to their semester plans.

The system will not require FTP or other file-transfer protocols for its normal operation because its main purpose is managing and displaying academic planning information through a web application.

## 4. Appendices

The appendices provide additional information to support the requirements described in this Software Requirements Specification (SRS). Appendix A presents the main database tables expected in the Academic Course & Schedule Planner, while Appendix B explains important technical and academic terms used throughout the document.

## 4.1 Appendix A: Data Dictionary

The following table describes the possible database tables that may be used to store and organize the system's information. These tables represent the expected database structure and may be adjusted during implementation.

| Table Name | Short Description |
|---|---|
| **Users** | Stores user account information, including names, email addresses, passwords, and account roles. |
| **Students** | Stores student-specific information, such as student ID and academic level. |
| **Administrators** | Stores information related to users with administrative privileges. |
| **AcademicPrograms** | Stores academic program information, including program names and total required credit hours. |
| **Courses** | Stores course details, including course IDs, names, descriptions, departments, and credit hours. |
| **Prerequisites** | Stores the relationships between courses and their required prerequisite courses. |
| **AcademicTerms** | Stores information about academic semesters, including term names and dates. |
| **Sections** | Stores course section offerings, including section numbers, capacities, available seats, and associated courses. |
| **MeetingTimes** | Stores the meeting days, start times, end times, and locations of course sections. |
| **SemesterPlans** | Stores semester plans created by students, including plan names, academic terms, and statuses. |
| **PlannedSections** | Stores the sections selected by students within their semester plans. |
| **CompletedCourses** | Stores students' completed courses, grades, completion dates, and earned credit hours. |
| **GradeScales** | Stores the grading systems used by the application for GPA calculations. |
| **GradeScaleEntries** | Stores grading details, including letter grades, grade points, and percentage ranges. |
| **DegreeRequirements** | Stores academic and graduation requirements, including required credit hours and requirement descriptions. |

**4.2 Appendix B: Glossary**

The following table defines important terms, abbreviations, and technical expressions used throughout this document.

| Term / Acronym | Definition |
|---|---|
| **Academic Course & Schedule Planner** | A web-based application that helps students organize semester plans, check prerequisites, build timetables, calculate GPA estimates, and monitor academic progress. |
| **Administrator** | An authorized user responsible for managing courses, sections, capacities, prerequisites, and grading rules. |
| **API** | Application Programming Interface. A set of rules that allows different software applications to communicate with each other. |
| **Authentication** | The process of verifying a user's identity before granting access to the system. |
| **Authorization** | The process of determining which features and information an authenticated user is allowed to access. |
| **Browser** | Software used to access and display websites, such as Google Chrome, Microsoft Edge, Firefox, and Safari. |
| **Cumulative GPA** | The overall Grade Point Average calculated using a student's academic results across multiple completed courses or semesters, according to the configured grading rules. |
| **Credit Hours** | A numerical value representing the academic workload or credit value assigned to a university course. |
| **Database** | An organized collection of information stored electronically and managed by a Database Management System. |
| **DBMS** | Database Management System. Software used to store, organize, retrieve, and manage data. |
| **Degree Requirements** | The academic conditions and credit-hour requirements that a student must satisfy to complete an academic program. |
| **Frontend** | The part of the application that users interact with directly through their web browsers. |
| **Backend** | The server-side part of the application responsible for processing requests, applying system logic, and communicating with the database. |

| Term / Acronym | Definition |
|---|---|
| **Functional Requirement (FR)** | A requirement that describes a specific function or behavior the system must provide. |
| **GPA** | Grade Point Average. A numerical representation of a student's academic performance based on course grades and credit hours. |
| **HTTPS** | Hypertext Transfer Protocol Secure. A communication protocol that encrypts information exchanged between a web browser and a web server. |
| **JSON** | JavaScript Object Notation. A lightweight data format commonly used to exchange information between software applications. |
| **Non-Functional Requirement (NFR)** | A requirement that describes a system's quality attributes, such as performance, security, reliability, and usability. |
| **Prerequisite** | A course or academic condition that must be completed or satisfied before a student is eligible to take another course. |
| **REST API** | Representational State Transfer Application Programming Interface. A common approach for communication between frontend and backend applications using HTTP methods. |
| **Section** | A specific offering of a course during an academic term, with its own section number, capacity, and meeting schedule. |
| **Semester Plan** | A collection of courses and sections selected by a student for a particular academic term. |
| **Student** | A user who accesses the system to explore courses, plan semesters, calculate GPA estimates, and monitor academic progress. |
| **Timetable** | A weekly schedule showing the days, times, and locations of a student's selected course sections. |
| **UML** | Unified Modeling Language. A standard visual language used to represent and design software systems. |
| **Use Case** | A description of how an actor interacts with the system to achieve a particular goal. |
| **Actor** | A person or external system that interacts with the application. |

| Term / Acronym | Definition |
|---|---|
| **Schedule Conflict** | An overlap between the meeting times of two or more selected course sections. |
| **Academic Progress** | An overview of a student's completed courses, earned credit hours, remaining requirements, and estimated progress toward graduation. |
| **Conceptual Class Diagram** | A UML diagram showing the main concepts or entities in a system and the relationships between them. |
| **Multiplicity** | A UML notation that indicates how many instances of one class can be associated with instances of another class. |
| **Data Integrity** | The accuracy, consistency, and reliability of information stored in a database. |